Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Вычисление функций с помощью рядов Тейлора»

Выполнил:
студент группы 3825Б1ПМии1
Митякин Р.С.

Проверил:
ст. преподаватель каф. ВВСП,
Волокитин В.Д.

Нижний Новгород
2026

Содержание

Постановка задачи....3
Метод решения....4
Руководство пользователя....6
Описание программной реализации....7
Подтверждение корректности....9
Результаты экспериментов....10
Заключение....11
Приложение....12

Постановка задачи

Целью данной лабораторной работы является разработка программы на языке C, вычисляющей значения функций sin(x), cos(x), exp(x) и ln(x) с помощью рядов Тейлора (Маклорена).

Программа должна удовлетворять следующим требованиям:

- Пользователь самостоятельно выбирает вычисляемую функцию и метод суммирования ряда;
- Каждая функция реализуется с помощью рекуррентного вычисления следующего слагаемого;
- Суммирование ряда выполняется одним из тремя способов: прямым, обратным и метод Кэхана;
- В коде используются структуры и указатели на функции;
- Результат сравнивается со значением из библиотеки math.h;

Метод решения

Ряд Маклорена - это частный случай ряда Тейлора, разложение функции в точке x = 0. Каждая из четырех функций раскладывается в следующие ряды.

$$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$$

$$\cos(x) = 1 - x^2/2! + x^4/4! - x^6/6! + \ldots$$

$$\exp(x) = 1 + x + x^2/2! + x^3/3! + \ldots$$

$$\ln(x) = 2\cdot[t + t^3/3 + t^5/5 + \ldots], \quad t = (x-1)/(x+1)$$

Для логарифма используется ряд через гиперболический арктангенс, поскольку он сходится при любом x > 0.

Вместо вычисления каждого слагаемого с нуля используется рекуррентное отношение - отношение соседних слагаемых. Это позволяет получить следующее слагаемое умножением предыдущего на некоторый множитель.

Функция	Первое слагаемое	Рекуррентное отношение a(n+1)/a(n)
sin(x)	x	$(-x^2) / ((2n+2)\cdot(2n+3))$
cos(x)	1	$(-x^2) / ((2n+1)\cdot(2n+2))$
exp(x)	1	$x / (n+1)$
ln(x)	2t	$t^2 \cdot (2n+1) / (2n+3)$

Методы суммирования:

1. Прямое суммирование - слагаемые накапливаются по порядку от первого к последнему. Простейший метод, но при большом числе слагаемых накапливает ошибку округления.

2. Обратное суммирование - все слагаемые сначала записываются в массив, затем суммируются от последнего к первому. Малые слагаемые складываются первыми, что уменьшает потери при сложении чисел разного масштаба.

3. Алгоритм Кэхана - компенсированное суммирование. На каждом шаге вычисляется и запоминается ошибка округления, которая компенсируется на следующем шагу. Алгоритм основан на формуле TwoSum:

```
c = a + b          // сложение с потерей
t = c - b          // часть a, сохранённая в c
d = a - t          // потерянная часть (передаётся дальше)
```

Для sin и cos при больших значениях x промежуточные слагаемые ряда становятся большими, а итоговый результат - малым. Это приводит к потере значащих цифр. Для уменьшения погрешности аргумент приводится к отрезку [0, 2π) с использованием периодичности функций.

```
count = (int)(x / 2π)
x = x − count · 2π
```

Для exp и ln редукция не применяется: exp не является периодической, а ln определен только при x > 0 и сходится при любых допустимых значениях аргумента.

Руководство пользователя

При запуске исполняемого файла, программа последовательно задает три вопроса:

1. Выбор вычисляемой функции:

```
1 — sin(x)
2 — cos(x)
3 - exp(x)
4 - ln(x)
Choose function:
```

2. Выбор метода суммирования:

```
1 — direct
2 — reverse
3 — Kahan
Choose method:
```

3. Ввод аргумента:

```
Enter x:
```

Программа выводит результат вычисления по ряду, значение из math.h и абсолютную погрешность. Пример вывода для sin(1), метод Кэхана:

```
1 — sin(x)
2 — cos(x)
3 - exp(x)
4 - ln(x)
Choose function: 1
1 — direct
2 — reverse
3 — Kahan
Choose method: 3
Enter x: 1

------------------------------
Result: 0.8414710
Value from math.h: 0.8414710
Error = 0.000000e+00
------------------------------
```

Ограничения: для ln(x) необходимо x > 0; для exp(x) при x > 88 результат равен inf, так как e^{88} находится у предела типа float.

Описание программной реализации

Проект реализован в виде единого файла *run.c*, который содержит все функции, методы суммирования и пользовательский интерфейс.

Для хранения состояния вычисления ряда используется структура

```
typedef struct {
    float term;
    int n;
    float x;
} TaylorTerms;
```

Поле term хранит значение текущего слагаемого, поле n - его порядковый номер, поле x - аргумент функции (редуцированный для sin и cos). Структура передается в функции перехода по указателю, что позволяет изменять ее поля напрямую.

Указатели на функции:

Указатель на функцию - переменная, хранящая адрес функции. Это позволяет передавать функцию как аргумент и вызывает ее внутри другой функции.

В программе используется один указатель next, который получает адрес нужной функции перехода в зависимости от выбора пользователя:

```
void (*next)(TaylorTerms *);
    if (func == 1) {
        next = next_sin;
        from_math_value = sinf(x);
    } else if (func == 2) {
        next = next_cos;
        from_math_value = cosf(x);
    } else if (func == 3) {
        next = next_exp;
        from_math_value = expf(x);
    } else if (func == 4) {
        next = next_ln;
        from_math_value = logf(x);
    } else {
        printf("Error.\n");
        return 1;
    }
```

Этот же указатель передается в функции суммирования, где вызывается как обычная функция: next(s). Таким образом все три метода суммирования работают с любой из четырех функций без дублирования кода.

Функции перехода:

Каждая функция перехода принимает указатель на TaylorTerms, умножает term на рекуррентный множитель и увеличивает n. Пример для sin:

```
void next_sin(TaylorTerms *s) {
    s->term *= -(s->x * s->x) / ((2.0f * s->n + 2) * (2.0f * s->n + 3));
    s->n++;
}
```

Все три функции суммирования принимают одинаковые аргументы: указатель на TaylorTerms и указатель на функцию перехода. Суммирование продолжается пока число шагов не превысило MAX_TERMS.

Подтверждение корректности

Проверка формул рядов

Корректность рядов проверяется сравнением с эталонными значениями из math.h. В таблице ниже приведены результаты функций, полученные с помощью алгоритма Кэхана при x = 1.

Функция	x	Результат по ряду	Эталон math.h	Погрешность
sin(x)	1.0	0.8414710	0.8414710	0.0e+00
cos(x)	1.0	0.5403023	0.5403023	5.9e-08
exp(x)	1.0	2.7182818	2.7182818	0.0e+00
ln(x)	1.0	0.0000000	0.0000000	0.0e+00

Проверка редукции

Если не делать редуцирование аргументов для sin и cos, то это приведет к потере точности. Ниже приведены сравнение значений sin(x) с редукцией к [0, 2π) и без нее.

x	Метод	Результат	Эталон	Погрешность
10.0	без редукции	-0.5440410	-0.5440211	1.99e-05
10.0	с редукцией	-0.5440211	-0.5440211	5.96e-08

Из таблицы видно, что редуцирование аргумента значительно повышает точность при вычислении.

Результаты экспериментов

Сравнение методов суммирования при малом x

При x = 1 (x = 2 в случае натурального логарифма) все три метода дают примерно одинаковую точность с минимальной погрешностью:

Функция	Прямое	Обратное	Кэхан
sin(1)	0.0e+00	0.0e+00	0.0e+00
cos(1)	0.0e+00	0.0e+00	5.9e-08
exp(1)	2.3e-07	0.0e+00	0.0e+00
ln(2)	0.0e+00	0.0e+00	0.0e+00

Сравнение методов суммирование при большом x:

При x = 20 разница между методами становится заметной. Слагаемые достигают больших значений, и накопление ошибок округления становится более заметным:

Функция	Прямое	Обратное	Кэхан
sin(20)	2.9e-07	2.3e-07	2.3e-07
cos(20)	4.4e-07	4.1e-07	4.4e-07
exp(20)	3.2e+01	6.4e+01	9.6e+01
ln(20)	4.7e-07	0.0e+00	0.0e+00

Прямое суммирование теряет все малые слагаемые при прибавлении к большому числу. Обратное улучшает результат, но при большом числе итераций само накапливает ошибку. Кэхан компенсирует ошибку на каждом шаге и дает точный результат.

Тип float обеспечивает около 7 значащих десятичных цифр. При больших x погрешность неизбежна даже с редукцией - например, при x = 1000 для sin погрешность достигает $1.5 \cdot 10^{-5}$ из-за потери точности при вычислении $x - count \cdot 2\pi$. При x > 88 для exp результат равен inf.

Заключение

В ходе лабораторной работы мне удалось реализовать программу на языке C, вычисляющую функции sin(x), cos(x), exp(x), ln(x) с помощью рядов Тейлора (Макларена). В программе реализованы:

- Четыре функции перехода к следующему слагаемому, основанные на рекуррентных соотношениях;
- Три метода суммирования - прямое, обратное и алгоритм Кэхана;
- Структура TaylorTerms для хранения состояние вычисления;
- Указатели на функции для универсального вызова функций перехода и суммирования;
- Редукция аргумента к [0, 2π) для sin и cos.

Эксперименты показали, что алгоритм Кэхана обеспечивает наилучшую точность при суммирования рядов с большим числом слагаемых разного масштаба. Обратное суммирование занимает промежуточное положение. Прямое суммирование наиболее простое, но накапливает наибольшую ошибку.

Основным источником погрешности при больших x является не метод суммирования, а потеря значащих цифр при вычитании близких чисел. Для sin и cos это проблема частично решается редукцией аргумента.

Приложение

```c
#include <stdio.h>
#include <math.h>

#define MAX_TERMS 1000000

typedef struct {
    float term;
    int n;
    float x;
} TaylorTerms;

void next_sin(TaylorTerms *s) {
    s->term *= -(s->x * s->x) / ((2.0f * s->n + 2) * (2.0f * s->n + 3));
    s->n++;
}

void next_cos(TaylorTerms *s) {
    s->term *= -(s->x * s->x) / ((2.0f * s->n + 1) * (2.0f * s->n + 2));
    s->n++;
}

void next_exp(TaylorTerms *s) {
    s->term *= s->x / (1.0f + s->n);
    s->n++;
}

void next_ln(TaylorTerms *s) {
    float t = (s->x - 1.0f) / (s->x + 1.0f);
    s->term *= (t * t) * (2.0f * s->n + 1) / (2.0f * s->n + 3);
    s->n++;
}

float forward_sum(TaylorTerms *s, void (*next)(TaylorTerms *)) {
    float sum = s->term;

    for (int i = 0; i < MAX_TERMS; i++) {
        next(s);
        sum += s->term;
    }

    return sum;
}

float back_sum (TaylorTerms *s, void (*next)(TaylorTerms *)) {
    float terms[MAX_TERMS];
    int index = 0;
    float sum = 0;

    terms[index] = s->term;
    index++;

    for (int i = 0; i < MAX_TERMS - 1; i++) {
        next(s);
        terms[index] = s->term;
        index++;
    }

    for (int i = index - 1; i >= 0; i--) {
         sum += terms[i];
    }

    return sum;
}
```

```c
float kahan_algo(TaylorTerms *s, void (*next)(TaylorTerms *)) {
    float sum = s->term;
    float d = 0;

    for (int i = 0; i < MAX_TERMS; i++) {
        next(s);

        float a = s->term + d;
        float b = sum;

        float c = a + b;
        float t = c - b;

        d = a - t;
        sum = c;

    }
    return sum;
}

float period(float x, int *sign) {
    if (x < 0.0f) {
        *sign = -1;
        x = -x;
    } else {
        *sign = 1;
    }

    int count = (int)(x / (2.0f * (float)M_PI));
    x = x - count * 2.0f * (float)M_PI;

    return x;

}

int main() {
    float x;
    int sign;
    int func;
    int algo;
    float new_x;
    float sum;
    float from_math_value;

    printf("1 — sin(x)\n");
    printf("2 — cos(x)\n");
    printf("3 - exp(x)\n");
    printf("4 - ln(x)\n");
    printf("Choose function: ");
    scanf("%d", &func);

    printf("1 — direct\n");
    printf("2 — reverse\n");
    printf("3 — Kahan\n");
    printf("Choose method: ");
    scanf("%d", &algo);

    printf("Enter x: ");
    scanf("%f", &x);

    sign = 1;
    if (func == 1 || func == 2) {
        new_x = period(x, &sign);
    } else {
        new_x = x;
    }
```

```c
    TaylorTerms s;
    s.n = 0;
    s.x = new_x;

    if (func == 1) {
        s.term = new_x;
    } else if (func == 4) {
        s.term = 2.0f * (x - 1.0f) / (x + 1.0f);
    } else {
        s.term = 1.0f;
    }

    void (*next)(TaylorTerms *);
    if (func == 1) {
        next = next_sin;
        from_math_value = sinf(x);
    } else if (func == 2) {
        next = next_cos;
        from_math_value = cosf(x);
    } else if (func == 3) {
        next = next_exp;
        from_math_value = expf(x);
    } else if (func == 4) {
        next = next_ln;
        from_math_value = logf(x);
    } else {
        printf("Error.\n");
        return 1;
    }

    if (algo == 1) {
        sum = forward_sum(&s, next);
    } else if (algo == 2) {
        sum = back_sum(&s, next);
    } else if (algo == 3) {
        sum = kahan_algo(&s, next);
    } else {
        printf("Error.\n");
        return 1;
    }

    if (func != 2) {
        sum *= sign;
    }

    printf("\n");
    printf("------------------------------\n");
    printf("Result: %.7f\n", sum);
    printf("Value from math.h: %.7f\n", from_math_value);
    printf("Error = %e\n", fabsf(sum - from_math_value));
    printf("------------------------------\n");
}
```